

# Late Breaking Results: PyAIE: A Python-based Programming Framework for Versal ACAP Platforms

**Abstract**—In order to satisfy the exploding demand from various computation-intensive applications, AMD-Xilinx released Versal, a heterogeneous Adaptive Compute Acceleration Platform (ACAP) with powerful heterogeneous acceleration capabilities. Most AI/ML applications use Python as their primary programming language these days; yet, AI engine codes as well as numerous Versal ACAP-based acceleration projects must be written in C/C++ with low-level intrinsics in order to take advantage of the Versal ACAP platform. To fill the gap of programming abstractions of applications and Versal ACAP platforms, we propose PyAIE, a Python-based programming framework specifically targeting Versal ACAP architecture. PyAIE allows users to focus on algorithm-level designs without knowledge of the underlying low-level details. PyAIE automatically translates Python code into the optimized C/C++ code for AI Engine kernels, PL kernels, and host, along with configuration script files, thereby completing the entire AI Engine-based system design. To the best of our knowledge, this is the first Python-based programming and compilation flow designed specifically for Versal ACAP platforms.

## I. INTRODUCTION

In the conventional Versal ACAP design flow, full knowledge of the functionalities of the heterogeneous Versal platform is required. Application code, AI Engine (AIE), programmable logic (PL) code, host code, and configuration files are all need to be written by developers. Manual development and maintenance of these codes for different level of architecture without automated tools is tedious and results in a long and error-prone development cycle. In order to overcome this inconvenience, we propose PyAIE, a programming framework that allows users to efficiently develop the Versal ACAP based systems with Python. PyAIE automates the entire process, reduces user burden, and increases productivity by eliminating the need to understand the low-level hardware details. Users would design algorithms in Python instead of C/C++, using PyAIE intrinsic APIs. PyAIE then divides the user-written tasks into subtasks and maps them onto different AIE cores, generating code for AIE cores and PL automatically.

The main contributions of this work can be summarized as follows.

- 1) To the best of our knowledge, our proposed flow is the first compilation flow to generate AIE kernel code, PL code, and host code from user's Python programs.
- 2) PyAIE compilation flow enables vectorization optimization by generating AIE built-in vector registers and instructions.
- 3) We demonstrate a prototype of Python-based system programming and tuning tool that optimizes computation and data movement on heterogeneous Versal ACAP platforms.

## II. RELATED WORKS

PyLog [1] is an algorithm-centric Python-based FPGA programming and synthesis flow, which served as inspiration for PyAIE. Users insert decorators (@pylog) into parts of the

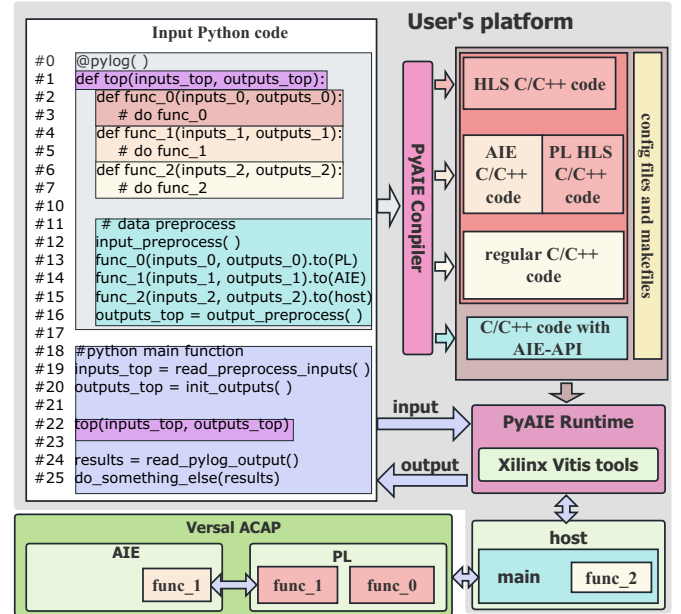


Fig. 1: PyAIE Framework Overview.

Python code, passing the decorated function into the PyLog process, which produces high-level synthesis (HLS) C code as an output. PyAIE can be regarded as an extension of PyLog for Versal ACAP platforms. Section III discusses the details of PyAIE design.

Since AMD-Xilinx's launch of the Versal ACAP, researchers have published numerous accelerator and optimizer designs based on this platform [2]–[4]; yet, these designs still require the manual effort of programmers. Recently, a new paper proposed an innovative matrix multiplication accelerator design that has a high utilization of the Versal ACAP architecture [5]. We incorporate this matrix multiplication accelerator into our flow and use it to implement generic tensor dot product operations. This work contains a template-based code generator, but it only focuses on matrix multiplication and it's not produced by Python. PyAIE specifically aims to improve the programmability and efficiency of Versal ACAP by providing a higher level of abstraction in the design flow and introducing an automated design process.

## III. PYAIE DESIGN

Fig. 1 is an overview of PyAIE framework. On the left-hand side is an example input Python code for the flow. From line 19 to 25, list the main function of this code. Firstly, line 19 prepares the inputs, which, e.g., includes specifying the data type and size, and storing the data in files for the host code to read. Line 20 initializes the data type and size of the output. Then the function `top` in line 22 is decorated with the Python decorator `@pylog` (line 0 and line 1), which passes the function as well as the arguments as an input to the PyAIE compiler. Finally, in line 24, the Python main function reads

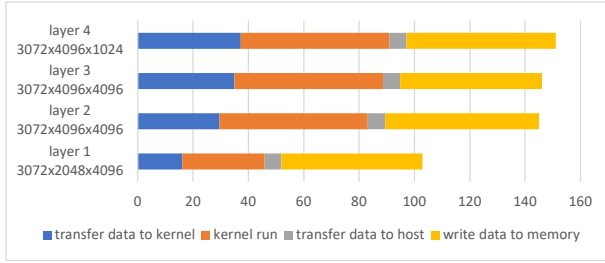


Fig. 2: Time Consumption of Layers in MLP (/ms).

the output files as the results of the function `top`, so that it can be used in the following portion.

Inside the `top` function, there are three sub-functions, `func_0`, `func_1` and `func_2` defined in line 2, line 4, and line 6 respectively. Line 12 does some preprocessing to the input data of the `top` function, and gets the inputs and outputs information for the three sub-functions. Line 13 to line 15 call those three sub-functions and indicate their targets by users. In this example, `func_0` is targeting PL, implying that PyAIE will only utilize the resources of the PL to implement its function. Similarly, `func_1` is supposed to target AIE; however, it will also use PL to implement the `data_mover` of AIE kernels to transfer the data between the AIE and host. Besides, if `func_1` is doing matrix multiplication, PL kernels are also needed to implement the function to arrange the results from AIE kernels [5]. Lastly, `func_2` will be targeted to the host only, which will be implemented as a sub-function of the host source code.

The input Python code will be passed to the PyAIE front-end analyzer, which will analyze the `top` function, get the architecture of the code, and the data flow between each sub-functions. For each sub-function (the PL, AIE and host subfunc arrows as shown in the central part of Fig. 1), it will pass the source code and the arguments to the `@pylog` front-end, and PyLog intermediate representation (PLIR) is generated. In the next step, the PyAIE compiler takes over and starts to generate kernels design targeting different portions of Versal ACAP. The `host` main arrow shows that the main function of `top` will be only passed to the PyAIE compiler to generate the `top` host source. In addition, the configuration files and all the Makefiles will be generated according to the data flow information.

After PyAIE generates all the needed files, PyAIE Runtime will take over and call the Vitis tools to compile the whole project to the device to execute it. Users can also choose to do this manually. The results are stored in files and can be processed by the Python main function. If the `top` function is called again, then only PyAIE Runtime will be activated and execute the existing binary file.

#### IV. OPTIMIZATIONS

In PyAIE, the computation of one-dimensional vector data in Python `numpy` is implemented with AIE matrix multiply APIs. For example, both `numpy.multiply` and `*` operations perform element-wise multiplication when the operands are one-dimensional vectors. In the generated AIE code, we call the AIE API `aie::mmul` to perform the vectorized computation for `numpy.multiply`, while for `*` operation, we generate for loops to complete the element-wise multiplication.

As for the optimization of the data-intensive operations such as matrix multiplication (`numpy.dot`, `numpy.matmul`), we integrated CHARM architecture [5] inside our framework. Upon encountering a large matrix multiplication, PyAIE will call the code generator of CHARM to generate the corresponding PL and AIE code. For a 4-layer MLP application with MM size  $3072 \times 2048 \times 4096$ ,  $3072 \times 4096 \times 4096$ ,  $3072 \times 4096 \times 4096$ , and  $3072 \times 4096 \times 1024$ , the total execution time of kernels running is 190ms, which is 8151x faster than Python `numpy.dot` operation. We list the time consumption of each layer in Fig. 2.

#### V. FUTURE WORKS

This is an on-going project that is being actively developed. Our future plan can be summarized as follows.

- 1) Further optimize generic data-intensive tensor operations in Python programs by improving the collaboration between AI Engine and programmable logic.
- 2) Optimize the system generation in the current PyAIE flow. By fully leveraging Python programming features, we would like to enable more flexible mapping from Python functions to heterogeneous computing elements inside Versal devices.
- 3) Improve the algorithm to allocate resources. Especially when there are multiple functions that need to be implemented on AIE. A suitable method needs to be found to determine the number of AIEs allocated to each function.
- 4) Closely integrate PyLog and PyAIE so that we can perform PL-AIE co-design and generate optimal PL accelerators and data movers for the corresponding AI Engine programs.

In summary, PyAIE is a Python-based programming flow that can generate system design for Versal ACAP platforms. The goal of the project is to improve the programming efficiency of Versal platforms by leveraging the expressiveness of Python language, the capabilities of compiler analysis and optimization, and the heterogeneity of Versal ACAP platforms. We plan to further improve the optimization passes inside PyAIE to generate better kernel-level and system-level designs.

#### REFERENCES

- [1] S. Huang, K. Wu, H. Jeong, C. Wang, D. Chen, and W.-M. Hwu, "Pylog: An algorithm-centric python-based fpga programming and synthesis flow," *IEEE Transactions on Computers*, vol. 70, no. 12, pp. 2015–2028, 2021.
- [2] C. Zhang, T. Geng, A. Guo, J. Tian, M. Herbordt, A. Li, and D. Tao, "H-gcn: A graph convolutional network accelerator on versal acap architecture," 2022. [Online]. Available: <https://arxiv.org/abs/2206.13734>
- [3] P. Chatarasi, S. Neuendorffer, S. Bayliss, K. Vissers, and V. Sarkar, "Vyasa: A high-performance vectorizing compiler for tensor convolutions on the xilinx ai engine," in 2020 IEEE High Performance Extreme Computing Conference (HPEC). Waltham, MA, USA: IEEE, 2020, pp. 1–10.
- [4] A. Al-Zoubi, G. Martino, F. H. Bahnsen, J. Zhu, H. Schlarb, and G. Fey, "Cnn implementation and analysis on xilinx versal acap at european xfel," in 2022 IEEE 35th International System-on-Chip Conference (SOCC). Belfast, United Kingdom: IEEE, 2022, pp. 1–6.
- [5] J. Zhuang, J. Lau, H. Ye, Z. Yang, Y. Du, J. Lo, K. Denolf, S. Neuendorffer, A. Jones, J. Hu, D. Chen, J. Cong, and P. Zhou, "Charm: Composing heterogeneous accelerators for matrix multiply on versal acap architecture," in The 2023 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, ser. FPGA '23, no. 12. New York, NY, USA: Association for Computing Machinery, 2023, pp. 1–12. [Online]. Available: <https://doi.org/10.1145/3543622.3573210>